Pricing Azerbaijani Real Estate from Scratch: Decision Trees and Pegasos SVMs on bina.az Listings

Aliyeva, Nurana Garibli, Sanan Jamalov, Humbat Nematzada, Farah Shukurov, Elsen

MLE-AI-201 Machine Learning, AI Academy — Cohort II

***Abstract*—**We predict real-estate prices from 100,775 bina.az sale listings with two models implemented from scratch in NumPy: a CART-style decision tree (Gini, entropy and MSE, vectorised split search, four stopping rules) and a soft-margin SVM trained with Pegasos, with random Fourier features as a non-linear extension. After deduplicating repeat scrapes, parsing text fields and removing target-derived columns (`unit_price`, `total_price`), 63,822 listings remain. We frame two tasks: regression on log price and a premium/standard tier defined by the training median (210,000 AZN). All hyperparameters are chosen on validation data, and the models are benchmarked against matched scikit-learn models with 5-fold cross-validation and one test evaluation. Our tree matches scikit-learn within one standard deviation (CV RMSE 0.338 vs. 0.337; CV F1 0.917 vs. 0.916) and is the best model overall, with test RMSE 0.332 on log price ($R^2 = 0.80$) and test F1 0.929. Our linear SVM matches scikit-learn's hinge-loss solvers (test F1 0.843), but a linear decision rule cannot express the interactions that drive price. Errors concentrate in the cheapest and most expensive listings and near the tier threshold. Code: `github.com/team-M503-SWE-06/ml_final_project` (tag `v1.0-final`).

***Index Terms*—**decision tree, support-vector machine, Pegasos, random Fourier features, regression, classification, real-estate price prediction.

I. Introduction

Online listings are the main price signal on the Azerbaijani housing market, but they are noisy: the same flat is re-posted many times, prices come in two currencies, and numbers arrive as free text. We use one such dump, the bina.az sale listings, to study two models that we implemented ourselves in NumPy and benchmark against scikit-learn [5].

From one dataset we frame two tasks. *Task A* is regression: predict the sale price, modelled as $\log(\text{price})$ because prices are heavy tailed. *Task B* is classification: decide whether a listing is *premium* (price above the training-set median) or *standard*. Our contributions are:

- a leakage-free cleaning pipeline and a frozen, stratified 70/15/15 split (Section III);
- a CART-style decision tree [1] for both tasks with Gini, entropy and MSE criteria, a vectorised split search and four stopping rules, tested node by node against scikit-learn;
- a Pegasos SVM [2] with margin and support-vector reporting, mini-batches, iterate averaging, and random Fourier features [3] as the non-linear extension;
- a protocol where every choice is made on validation data, a 5-fold CV benchmark against matched scikit-learn models, one final test evaluation, and an error analysis of where the models fail.

Everything is reproduced by one command, `python -m src.run_all`.

II. Background

Decision trees. A regression or classification tree recursively splits the data with axis-aligned questions $x_j \le t$. Each split is chosen to maximise the impurity decrease

$$\Delta I = I(\text{parent}) - \tfrac{N_L}{N} I(L) - \tfrac{N_R}{N} I(R), \qquad (1)$$

with Gini $1 - \sum_k p_k^2$ or entropy $-\sum_k p_k \log_2 p_k$ for classification and the variance $\frac{1}{N}\sum_i (y_i - \bar{y})^2$ for regression [1]. Deep trees have low bias and high variance, so the depth or the node sizes must be controlled.

Soft-margin SVM and Pegasos. A linear SVM minimises the regularised hinge loss

$$J(\mathbf{w}, b) = \tfrac{\lambda}{2}\|\mathbf{w}\|^2 + \tfrac{1}{n}\sum_{i=1}^{n} \max\big(0,\, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\big), \qquad (2)$$

whose margin width is $2/\|\mathbf{w}\|$. Pegasos [2] solves (2) by stochastic sub-gradient steps with learning rate $\eta_t = 1/(\lambda t)$, so no quadratic-programming solver is needed. Random Fourier features [3] map $\mathbf{x}$ to $\phi(\mathbf{x})$ with $\phi(\mathbf{x})^\top \phi(\mathbf{x}') \approx \exp(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2)$, so a linear solver on ϕ approximates an RBF-kernel SVM.

III. Data and Preprocessing

We use the Kaggle *Bina.az Sale Project* dump (`sehriyarmemmedli/binaaz-sale-project`, file `house_sale.csv`, 100,775 rows × 51 columns). Every cleaning decision is logged by `data_prep.clean` and printed by `run_all`:

- **Duplicates.** 36,321 rows are repeat scrapes of the same listing (same URL); we keep the latest. No exact duplicates remain.
- **Currency and types.** All remaining prices are in AZN (the USD path converts at 1.70). Area strings such as "85 m^2" and "6 sot" (1 sot = 100 m^2) and floors such as "3 / 9" are parsed into numbers (`floor`, `total_floors`).
- **Outliers.** We drop prices below 1,000 AZN or outside a fixed 10,000–5,000,000 AZN range (457 rows), impossible areas and per-category area outliers (175 rows), and set coordinates outside Azerbaijan's bounding box

Table 1. Dataset summary after cleaning.

Property	Value
Raw rows / columns	100,775 / 51
Rows after cleaning	63,822
Features used	72
Train / Val / Test	44,676 / 9,573 / 9,573
Tier threshold (train median)	210,000 AZN
Premium share (each split)	49.3%

to missing. Limits are fixed domain values, not data quantiles, so they cannot leak test information.

- **Leakage.** `unit_price` (= price / area) and `total_price` (= price in 99.99% of rows) are arithmetic functions of the target and are removed, together with identifiers (`owner_name`, `shop_name`, `address`, URLs, ids), free text and scrape metadata (views, timestamps).
- **Missing values.** Medians are computed on the training rows only and every imputed column gets a 0/1 `_missing` flag (`land_area` is 85% missing, mostly flats).

The clean table has 63,822 listings. The feature matrix has 72 columns: 7 numeric (area, rooms, floor, total floors, land area, latitude, longitude), 6 missing-value flags, 6 binary flags (repair, bill of sale, mortgage, agent, residential complex, Baku) and 53 one-hot levels for category, district and location; rare levels learned on the training rows are pooled into "other".

The split is a seeded, stratified 70/15/15 partition frozen to `data/split_indices.npz` (Table 1). The tier label is *premium* when the price exceeds the training-set median of 210,000 AZN; validation and test labels reuse that threshold, so the test set never influences it. The resulting classes are balanced (49.3% premium in every split).

IV. Method

A. Decision tree (from scratch)

`src/decision_tree.py` implements one `DecisionTree` class for both tasks (Gini, entropy or MSE).

Split search. At each node every feature is sorted once. A threshold between sorted positions i and $i+1$ sends the first $i+1$ rows left, so cumulative sums over the sorted rows give the class counts (classification) or $\sum y$ and $\sum y^2$ (regression) of every left child at once; the right child is "node total minus left". This scores all candidate thresholds of all features in one vectorised pass, costing $O(nd \log n)$ per node instead of $O(n^2 d)$. Candidates are midpoints between consecutive *distinct* values, as in scikit-learn.

Growth and stopping. The tree grows by recursive partitioning (driven by an explicit stack, so very deep trees cannot hit Python's recursion limit). A node becomes a leaf when it reaches `max_depth`, has fewer than `min_samples_split` rows, is pure, has no threshold leaving `min_samples_leaf` rows on each side, or when the weighted decrease $(N_t/N)\,\Delta I$ is below `min_impurity_decrease` (scikit-learn's definition). Leaves store class frequencies (majority vote and `predict_proba`) or the mean. Feature importances are the normalised total weighted impurity decrease.

Ties and validation. When two features give the same gain we take the lowest feature index; scikit-learn breaks such ties at random. The tree rejects NaN or infinite inputs, checks the column count at predict time, and computes thresholds as $a/2 + b/2$ to avoid overflow.

Tests. `tests/test_decision_tree.py` compares our tree node by node (feature, threshold, samples, value) with scikit-learn on a toy set and on seeded random data for all three criteria; each example is first checked to give the same scikit-learn tree for ten random seeds, so the expected tree is fully determined. Further tests check that each stopping rule changes the tree, that `predict_proba` sums to one, and the input validation.

B. Support-vector machine (from scratch)

`src/svm.py` implements `PegasosSVM`. Labels are mapped to ± 1; the bias is treated as the weight of a constant-1 feature, so the objective actually minimised is (2) with $\frac{\lambda}{2}(\|\mathbf{w}\|^2 + b^2)$, a negligible change on standardised features that keeps the very large first steps ($\eta_1 = 1/\lambda$) stable. Each step over a mini-batch A_t of size k with active set $A_t^+ = \{i : y_i(\mathbf{w}^\top \mathbf{x}_i + b) < 1\}$ is

$$\mathbf{w} \leftarrow (1 - \eta_t \lambda)\,\mathbf{w} + \tfrac{\eta_t}{k} \textstyle\sum_{i \in A_t^+} y_i \mathbf{x}_i, \tag{3}$$

with three schedules: Pegasos $1/(\lambda t)$, $\eta_0/\sqrt{t}$ and constant. An optional projection onto $\|(\mathbf{w}, b)\| \le 1/\sqrt{\lambda}$ follows each step, and the returned model averages the iterates of the final epoch (tail averaging), which removes most of the step noise. The objective is logged every epoch, and the model reports $\|\mathbf{w}\|$, the margin width $2/\|\mathbf{w}\|$ and the support vectors ($y_i f(\mathbf{x}_i) \le 1$). Features are standardised with training statistics first, because the hinge loss is scale sensitive.

Non-linear extension. We chose random Fourier features (`RFFPegasosSVM`): $\phi(\mathbf{x}) = \sqrt{2/D}\cos(W^\top \mathbf{x} + \mathbf{c})$ with $W_{jk} \sim \mathcal{N}(0, 2\gamma)$, $c_k \sim U(0, 2\pi)$ and $D = 500$, followed by the unchanged linear solver. Kernelised Pegasos would need $O(n^2)$ kernel evaluations per epoch (about 2×10^9 for our 44,676 training rows) and $O(n)$ work per prediction; RFF costs $O(nD)$ per epoch and reuses the tested linear code.

V. Experimental Setup

Metrics. Task A: RMSE, MAE and R^2 on log(price), plus RMSE and MAE in AZN. Because $\exp(\widehat{\log y})$ estimates the median rather than the mean, AZN predictions are rescaled by Duan's smearing factor [4], estimated on validation residuals. Task B: precision, recall, F1 (premium is the positive class), ROC-AUC and the confusion matrix.

Protocol. Hyperparameters are chosen on the validation split only: depth, criterion and leaf-size sweeps for the tree, a 5-fold CV search on the training split for the final tree settings, and λ, epoch and schedule sweeps (and the RFF bandwidth γ) for the SVM. The benchmark then reports 5-fold CV mean $\pm$ std on the training split and evaluates every model once on the test split. Every random step uses seed 42 and the split is frozen, so reruns give identical numbers.

Baselines. scikit-learn models with matched settings: `DecisionTreeRegressor/Classifier` with the same hyperparameters; for the SVM, `SGDClassifier` (hinge, $\alpha =$

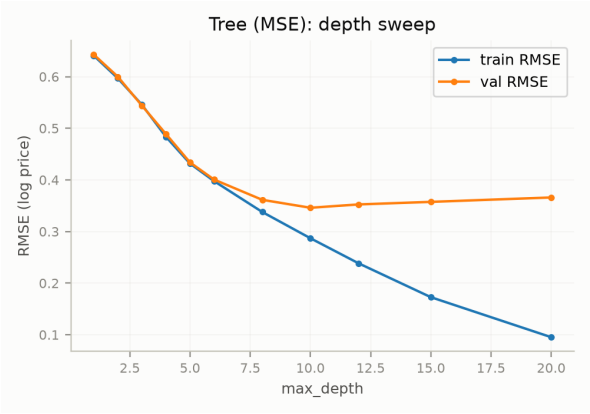


Figure 1. Regression tree, depth sweep (validation): the gap between training and validation RMSE opens after depth 10.

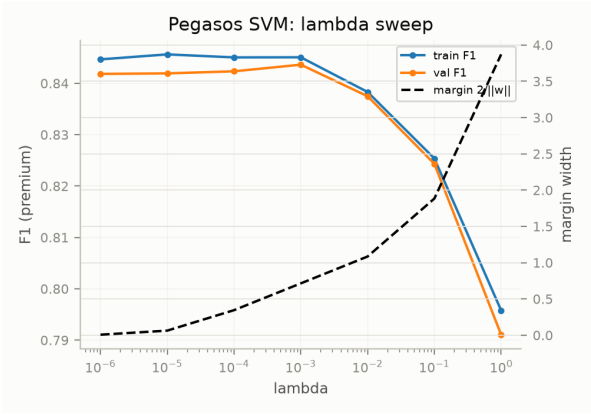


Figure 2. Pegasos SVM, λ sweep (validation): stronger regularisation widens the margin but costs accuracy beyond $\lambda = 10^{-3}$.

λ) and `LinearSVC` (hinge, $C = 1/(\lambda n)$), both with the scaler inside a `Pipeline`, and an RBF `SVC` on a 10,000-row training subsample (SVC is $O(n^2)$).

VI. Results

A. Tuning the tree

Fig. 1 shows the bias–variance trade-off of the regression tree: training RMSE keeps falling with depth, while validation RMSE is lowest at depth 10 (0.346) and rises again (0.366 at depth 20, where the training RMSE is 0.095). For the tier task, Gini and entropy are within 0.003 F1 at every depth; Gini peaks at depth 15 (validation F1 0.919), and larger `min_samples_leaf` values only lowered validation F1.

A single depth limit cuts every branch at the same level, although dense districts need deeper trees than sparse ones. A 5-fold CV search on the training split over all four stopping rules therefore replaced it with node-size rules: `min_samples_split=75` with no depth limit for Task A (CV RMSE $0.3621 \pm 0.0033 \rightarrow 0.3377 \pm 0.0032$) and Gini with `max_depth=20` and `min_impurity_decrease=2` $\cdot$ 10^{-5} for Task B (CV F1 $0.9115 \pm 0.0027 \rightarrow 0.9166 \pm 0.0019$). Both gains hold on every fold, and these are the configurations used below.

B. Tuning the SVM

On standardised features, validation F1 of the linear Pegasos SVM is flat for $\lambda \in [10^{-6}, 10^{-3}]$ (0.842–0.844) and drops as the regularisation grows (0.791 at $\lambda = 1$), while the margin $2/\|\mathbf{w}\|$ widens from 0.007 to 3.87 and the number of support vectors rises from 7,081 to 36,837 (Fig. 2). We keep $\lambda = 10^{-3}$ (F1 0.844, margin 0.71). The objective decreases over the epochs; the Pegasos and $\eta_0/\sqrt{t}$ schedules converge to the same objective (about 0.385 after 50 epochs), while a constant step of 0.01 oscillates and stays near 0.45. Averaging the last epoch reaches, in 10 epochs, the objective that the last iterate needs about 50 epochs for (0.385 vs. 0.395 at 10 epochs). Our weight vector points in almost the same direction as scikit-learn's solutions (cosine 0.997 with both `SGDClassifier` and `LinearSVC`). Random Fourier features did not help: the best bandwidth ($\gamma = 0.01$, $D = 500$) reached validation F1 0.809, below the linear model.

Table 2. 5-fold CV on the training split (mean $\pm$ std) and fit time per fold.

Task	Model	Score	Fit (s)
A (RMSE)	Ours: tree	0.3377 ± 0.0032	6.87
	sklearn tree	0.3369 ± 0.0038	0.74
B (F1)	Ours: tree	0.9166 ± 0.0019	17.50
	sklearn tree	0.9162 ± 0.0030	0.77
	Ours: Pegasos SVM	0.8433 ± 0.0029	8.76
	sklearn SGD (hinge)	0.8404 ± 0.0036	0.60
	sklearn LinearSVC	0.8416 ± 0.0022	1.83
	sklearn RBF SVC†	0.8497 ± 0.0091	5.43

† 10,000-row subsample per fold.

C. Benchmark against scikit-learn

Table 2 gives 5-fold CV on the training split and Table 3 the single test evaluation. Our tree matches scikit-learn's within one CV standard deviation on both tasks. Our SVM is on par with the matched linear baselines (CV F1 0.843 vs. 0.840 and 0.842); only the RBF `SVC`, trained on a 10,000-row subsample, is slightly higher (0.850 ± 0.009). On the test set the tree reaches an RMSE of 0.332 on log price ($R^2 = 0.80$) and F1 0.929 on the tier task, against F1 0.843 for the SVM.

The trees are structurally close to scikit-learn's: for Task B both have depth 20 and 1,655 vs. 1,661 leaves, and 99.1% of test predictions are identical; for Task A (no depth limit) the depths are 38 vs. 33 with 1,507 vs. 1,497 leaves and 94.2% identical predictions. The gaps come from tie-breaking (we take the lowest feature index; scikit-learn draws features at random) and from scikit-learn storing thresholds in float32; on tie-free examples our unit tests reproduce its trees node by node. Speed is the main difference: our tree fits in 6.9 s (Task A) and 17.5 s (Task B) per fold against 0.7–0.8 s, and our SVM in 8.8 s against 0.6 s for `SGDClassifier`, because the split search and the Pegasos steps run in Python/NumPy instead of compiled code.

D. Feature importance and SVM weights

The tree's impurity-based importances are dominated by floor area, followed by location (longitude, latitude and the pooled "other" location), land area and the category (land, house, new build). The SVM weights (standardised fea-

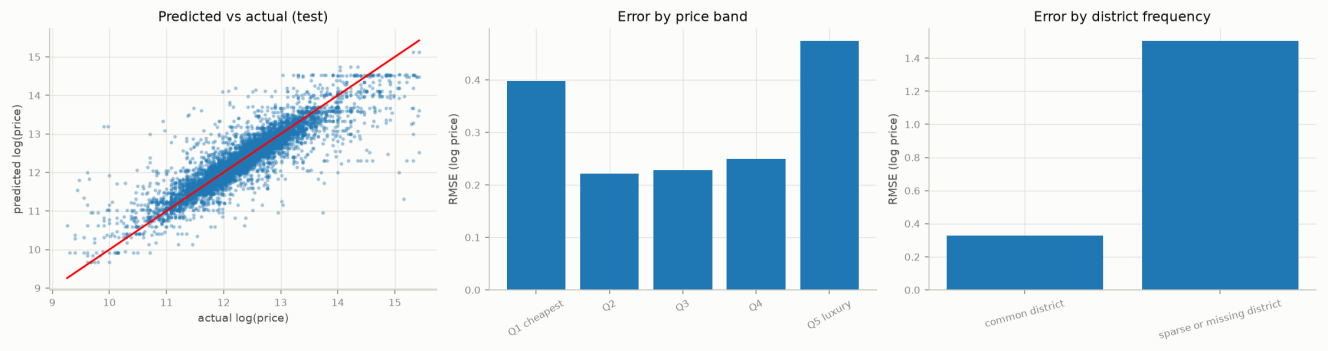


Figure 3. Test-set error analysis of the final regression tree: predictions vs. actual log price, RMSE by price quintile, and RMSE by district frequency (the sparse group has only nine listings).

Table 3. Headline results on the test set (one evaluation).

Task A: log(price)				
Model	RMSE	MAE	R^2	
Ours: tree	0.332	0.193	0.802	
sklearn tree	0.332	0.193	0.802	
Task B: price tier (premium = positive)				
Model	Prec.	Recall	F1	AUC
Ours: tree	0.926	0.932	0.929	0.925
sklearn tree	0.927	0.933	0.930	0.927
Ours: SVM	0.823	0.865	0.843	0.922
sklearn SGD	0.828	0.849	0.839	0.921
sklearn LinearSVC	0.828	0.856	0.842	0.921

Table 4. Test errors of the final trees by price quintile.

		Task A		Task B
Quintile	n	RMSE	med. err. (%)	error rate
Q1 cheapest	1,851	0.398	13.5	0.029
Q2	1,993	0.221	8.9	0.045
Q3	1,862	0.228	8.7	0.203
Q4	1,858	0.250	10.7	0.062
Q5 luxury	2,009	0.474	19.2	0.017

tures) tell a consistent story: the largest positive weights are rooms, area, the missing-rooms flag (typical of land plots and houses), *new build*, and central districts and neighbourhoods (Sabail, White City, Shah Ismail Khatai); the most negative are peripheral districts (Khachmaz, Absheron, Surakhani), *land* and *old build*.

E. Error analysis

Fig. 3 and Table 4 break the test errors of the final trees down. Task A errors are U-shaped across price quintiles: the middle three quintiles have an RMSE of 0.22–0.25 (median error 9–11%), but the cheapest quintile reaches 0.40 and the most expensive 0.47. In the top 5% of prices the RMSE is 0.75 and the median error is 44%. The ten largest errors are mostly land plots whose prices range from 20,000 to 5,000,000 AZN for similar areas and districts, which the features cannot separate. Task B errors concentrate at the tier boundary: listings within 10% of the 210,000 AZN threshold are misclassified 25.4% of the time, against 4.8% for the rest, and the middle price quintile has a 20.3% error rate against 1.7–6.2% elsewhere. Only nine test listings fall in sparse or missing districts; their RMSE is high (1.50) but too few to draw conclusions from.

VII. Discussion and Limitations

Why the tree beats the SVM. Price depends on interactions such as area × district × category, which a tree models with nested splits. The linear SVM can only add up per-feature contributions, so it plateaus at F1 0.84 regardless of λ, and its errors do not shrink with more epochs. Random Fourier features should capture interactions, but with $D = 500$ components over 72 mostly binary features the approximation was too coarse and underperformed the linear model; a kernel SVC on 10,000 rows gained less than one point. A tree, in contrast, overfits easily, which is why the node-size rules mattered.

Where the models fail. Both extremes of the price range are hard. A leaf predicts the mean log price of similar listings, so predictions are pulled towards the middle: cheap listings are over-estimated and luxury ones under-estimated. On top of this, $\exp(\widehat{\log y})$ estimates the median, not the mean (Jensen's inequality); the smearing factor (1.059) corrects the average level in AZN but cannot fix the tails. The largest single errors are land plots, whose price depends on things the data does not describe well (zoning, exact location, road access). For the tier task, most mistakes are listings priced close to the threshold, which is expected for a hard 210,000 AZN cut.

Limitations. (i) Errors in AZN remain large (test RMSE 253,669 AZN, MAE 79,550 AZN) because a few multi-million listings dominate the squared error; the log scale is the fairer summary. (ii) The data is a single scrape of asking prices, not transaction prices, and repeated listings had to be deduplicated by URL. (iii) One-hot encoding lets each split separate only one district from the rest; latitude and longitude partly compensate. (iv) The 5-fold CV search that chose the final tree settings is not part of `run_all`; the benchmark re-runs CV for the chosen settings only. (v) Our implementations are 9–23× slower to fit than scikit-learn's compiled ones. (vi) The SVM regularises the bias as a constant feature, a small departure from the textbook objective.

VIII. Conclusion

A from-scratch NumPy decision tree and Pegasos SVM, trained on a cleaned, leakage-free version of the bina.az listings, match scikit-learn within one CV standard deviation. The tree is the better model for both tasks: test RMSE 0.332 on log price ($R^2 = 0.80$) and F1 0.929 on the price tier, against F1 0.843 for the linear SVM. The remaining errors sit at the price extremes and at the tier boundary. Next steps are ensembles of our own trees to reduce the variance of a single tree, target or ordered encoding for districts, and richer location features.

Tools and Acknowledgements

We used the Kaggle *Bina.az Sale Project* dataset, NumPy, pandas, Matplotlib, and scikit-learn for the baselines only. AI assistance: Claude (Anthropic) was used to test the initial versions drafted according to each member's role. All code was reviewed in pull requests and checked against scikit-learn.

References

[1] L. Breiman, J. Friedman, R. Olshen, and C. Stone, *Classification and Regression Trees*. Wadsworth, 1984.

[2] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter, "Pegasos: primal estimated sub-gradient solver for SVM," *Mathematical Programming*, vol. 127, no. 1, pp. 3–30, 2011.

[3] A. Rahimi and B. Recht, "Random features for large-scale kernel machines," in *Advances in Neural Information Processing Systems*, 2007.

[4] N. Duan, "Smearing estimate: a nonparametric retransformation method," *Journal of the American Statistical Association*, vol. 78, no. 383, pp. 605–610, 1983.

[5] F. Pedregosa *et al.*, "Scikit-learn: machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.